



# *Learning to Simulate Cloth: Physics-Informed Deep Learning for Deformable Objects*

## State of the Art Report

15/01/2026

Student : **Antonin GRANADOS**  
[antonin.granados@telecom-paris.fr](mailto:antonin.granados@telecom-paris.fr)  
Supervisor : **Kiwon UM**  
[kiwon.um@telecom-paris.fr](mailto:kiwon.um@telecom-paris.fr)

School : **Télécom Paris**  
Course : **MDC-4EL01-TP (RECH-201)**

### Abstract

Physics-based simulation of deformable objects, such as cloth, plays a central role in computer graphics but remains computationally demanding for applications requiring real-time or highly detailed motion. Recent advances in deep learning, especially physics-informed graph neural networks (GNNs), offer a compelling alternative by learning deformation dynamics directly from synthetic simulation data. This report reviews the main methodologies underlying these approaches, with a focus on MeshGraphNet (MGN) and its subsequent extensions for improved long range interaction handling, collision response, and temporal stability. We further examine the datasets and learning paradigms used to train these models, including supervised, semi-supervised, and self-supervised strategies grounded in physical constraints. Despite significant progress, several challenges persist: accumulated errors over long rollouts, limited generalisation to unseen materials or mesh topologies, strong dependence on large high-quality synthetic datasets, and scalability issues for high-resolution garments. This survey analyses these limitations and outlines key research directions for enabling robust, efficient, and physically plausible cloth simulation in future graphics applications.

# Contents

|                                                                              |          |
|------------------------------------------------------------------------------|----------|
| <b>1. Physics-informed Learning</b>                                          | <b>1</b> |
| 1.1. Meshes in Cloth Simulation . . . . .                                    | 1        |
| 1.2. Core Architecture in Data-Driven Methods . . . . .                      | 2        |
| <b>2. Dataset and Evaluation</b>                                             | <b>3</b> |
| 2.1. Common Datasets . . . . .                                               | 3        |
| 2.2. Self-supervised and Unsupervised Learning . . . . .                     | 4        |
| <b>3. Challenges and Limitations</b>                                         | <b>5</b> |
| 3.1. Incomplete Collision Handling . . . . .                                 | 5        |
| 3.2. Temporal Instability and Extrapolation to Unseen Dynamics . . . . .     | 5        |
| 3.3. Scalability Limits and Supervision Requirements . . . . .               | 6        |
| 3.4. Trade-off Between Detail Preservation and Physical Guarantees . . . . . | 6        |
| <b>Acronyms</b>                                                              | <b>7</b> |
| <b>Glossary</b>                                                              | <b>7</b> |
| <b>A Results Replication</b>                                                 | <b>8</b> |
| A1. MeshGraphNets [11] . . . . .                                             | 8        |
| A2. Experiments with Alternative Learning Methods . . . . .                  | 8        |

# Introduction

The simulation of deformable objects, especially cloth, is central to computer graphics applications. Over the years, realistic cloth behaviour has been critical in film animation, video games, virtual reality, and more recently in virtual try-on for fashion. This problem has been researched for multiple decades, and a wide range of methods have been proposed to address it.

The most commonly used methods are physics-based. They rely on numerical solvers (such as ARCSim [10, 9]) to integrate elastic energies and enforce bending or stretching constraints. While these methods can provide realistic motion, they require detailed objects and precise parameter fine-tuning. They are also often computationally expensive, making them unusable in an interactive or real-time context and restricting them to offline computation.

But recently, deep learning approaches have emerged as a promising alternative. These methods leverage neural network architectures to learn the physical deformations of cloth from data. They promise orders of magnitude faster inference as well as strong generalisation capabilities, enabling their use in online applications and removing the need for complex and slow physics engines.

This report provides a comprehensive overview of the state of the art in physics-informed deep learning methods for cloth simulation. It covers key methodologies such as graph neural networks (GNNs) on mesh objects and physics-based loss functions, discusses usual datasets, and identifies the main technical bottlenecks that limit current performance.

## 1. Physics-informed Learning

### 1.1. Meshes in Cloth Simulation

3D meshes are fundamental in computer graphics applications, serving as a standard discretization to represent 3D objects and scenes. This is because they are designed to be easily and efficiently processed by GPUs. This inherent structure makes them ideal for simulations intended for computer graphics applications.

Beyond their role as a geometric representation, meshes are also at the core of most physics-based simulation methods for deformable objects, such as mass-spring systems and finite element methods (FEMs). In classical cloth solvers, each vertex stores state variables of the system (e.g. the position of a point on the object), while edges and faces define local physical interactions such as stretching, bending, or collision. These interactions are formulated through energy or force models, whose contributions are accumulated at each vertex and integrated over time to update the mesh.

The direct counterpart of these methods in learning-based simulation is GNNs, which have become a popular choice for learning the physical dynamics of mesh-based structures. The vertices of the object are modelled as nodes in a graph, and mesh connectivity defines the graph edges. One of the most important approaches proposed is MeshGraphNet (MGN) [11]. Their models are not only able to simulate cloth dynamics but also a wide variety of physical phenomena, including fluid dynamics [8] and solid mechanics. The success of this architecture lies in its message-passing framework, where information propagates through the graph structure to model complex physical interactions. Instead of computing the energies and forces of each interaction, the model learns the update rules from data.

Multiple other approaches have been explored to leverage meshes in deep learning models. One family of methods is presented in the following articles: [1, 14, 12, 13]. They use the skeletal representation of a human body as input and predict cloth deformation based on body pose, which is then skinned onto the 3D model. However, this is limited to virtual try-on applications and cannot be easily extended to more general simulations. Another type of architecture uses Fourier neural operators to learn the cloth motion [3]. This approach has the advantage of being resolution-agnostic, meaning

that the same model can operate on meshes of varying resolutions without retraining, unlike most GNN-based architectures. But they struggle to handle arbitrary mesh topologies such as complex garments.

## 1.2. Core Architecture in Data-Driven Methods

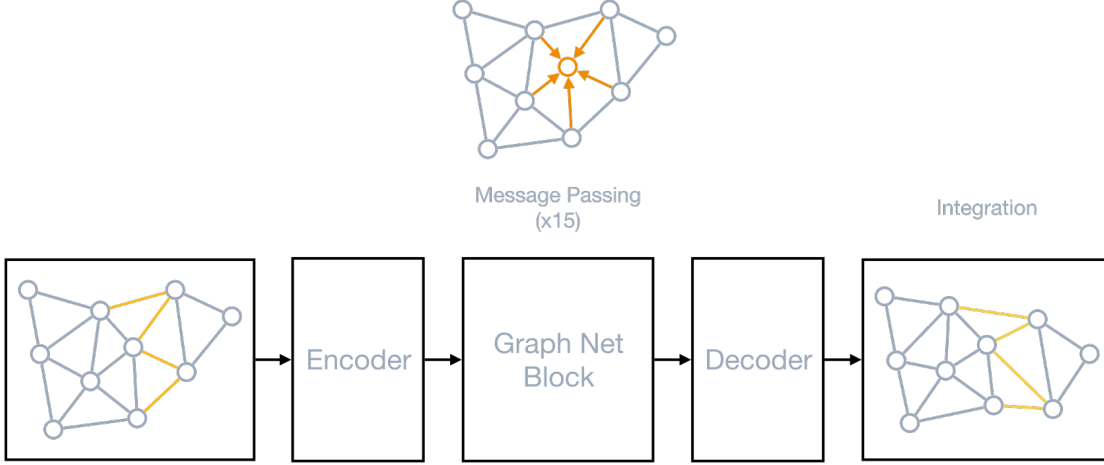


Figure 1: Representation of the MGN architecture, the yellow edges represent *world edges* and the grey edges are the *mesh edges* (adapted from MGN [11])

Many of the approaches that came after MGN [11] built upon its foundation, with many adapting its core architecture for specialised applications. These developments include improved handling of long range dependencies [4], better collision response [6], and enhanced temporal stability [7]. The graph-based representation is particularly well suited to cloth simulation, as it naturally accommodates irregular mesh structures and enables local interactions to be modelled directly on the discretised surface.

MGN represents a simulation state as a graph  $G = (V, E)$ , where  $V$  is the set of nodes corresponding to mesh vertices and  $E$  is the set of edges defining neighbourhood relationships. This graph represents the cloth mesh at a given time step. In addition to standard mesh edges, some architectures [11, 6] introduce *world edges* (depicted in yellow in 1), which connect vertices that are close in world space but not adjacent in the mesh. These edges enable the model to account for contact and collision interactions that cannot be captured by mesh connectivity alone.

The model follows an encoder-decoder architecture with intermediate message-passing steps. Node and edge features are first mapped to a latent space, then iteratively updated through neighbourhood aggregation, and finally decoded into per-vertex accelerations. These accelerations are subsequently integrated using a time stepping scheme to advance the cloth state.

We denote by  $e_{ij}^K$  the feature vector associated with the edge connecting vertices  $i$  and  $j$ , which encodes relative geometric information between the two vertices as well as edge-specific physical quantities (e.g. rest length). Similarly,  $v_i$  denotes the feature vector of vertex  $i$ , containing its local dynamic state, such as velocity or type indicators. Superscripts  $M$ ,  $W$ , and  $V$  respectively refer to mesh edges, world edges, and vertices. The functions  $\varepsilon^K$ ,  $f^K$ , and  $\delta^K$  are implemented as multi-layer perceptron (MLP)s. The update rules are defined as follows:

- Encoding:

$$e_{ij}^M \leftarrow \varepsilon^M(e_{ij}^M), \quad e_{ij}^W \leftarrow \varepsilon^W(e_{ij}^W), \quad v_i \leftarrow \varepsilon^V(v_i)$$

- Message Passing:

$$e_{ij}'^M \leftarrow f^M(e_{ij}^M, v_i, v_j), \quad e_{ij}'^W \leftarrow f^W(e_{ij}^W, v_i, v_j), \quad v_i' \leftarrow f^V\left(v_i, \sum_j e_{ij}'^M, \sum_j e_{ij}'^W\right)$$

- Decoding:

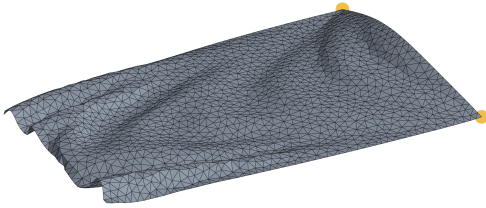
$$e_{ij}^M \leftarrow \delta^M(e_{ij}^M), \quad e_{ij}^W \leftarrow \delta^W(e_{ij}^W), \quad v_i \leftarrow \delta^V(v_i)$$

In the context of cloth simulation, these update rules define a learned operator that propagates and aggregates local information over the mesh to produce a single simulation step. Mesh edges constrain interactions to the cloth topology, while world edges enable information exchange between spatially proximal vertices, which is necessary for collision handling. After several message-passing iterations that allow for relatively long range interactions, the decoded vertex accelerations are integrated over time, yielding the updated cloth configuration at the next time step. We can describe this formally with the operator  $\mathcal{F}_\theta$ , using the position  $\mathbf{x}$ , the velocity  $\mathbf{v}$  and acceleration  $\mathbf{a}$  at time  $t$  and  $t + 1$  with time step  $\Delta t$ :

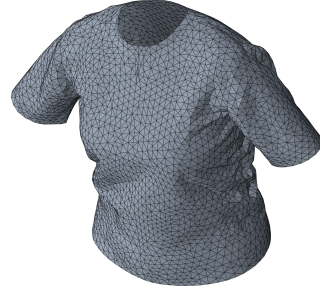
$$\begin{aligned} \mathbf{a}_t &= \mathcal{F}_\theta(\mathbf{x}_t, \mathbf{v}_t, G) \\ \mathbf{v}_{t+1} &= \mathbf{v}_t + \Delta t \mathbf{a}_t \\ \mathbf{x}_{t+1} &= \mathbf{x}_t + \Delta t \mathbf{v}_{t+1} \end{aligned}$$

## 2. Dataset and Evaluation

### 2.1. Common Datasets



(a) ARCSim: flag in the wind



(b) VTO dataset: t-shirt on a woman body

Figure 2: Frame from two different datasets

Learning cloth simulation models requires large datasets capturing a wide diversity of fabrics, motions, geometries, and collision events. Since collecting real-world ground-truth cloth motion is extremely difficult, most datasets rely on synthetic generation using physics-based simulators.

Most papers here use synthetic data generated using the ARCSim simulator [10, 9]. This simulation engine allows for the generation of many deformable materials such as cloth, paper, plastic, and metal. It also supports remeshing, which can help with the creation of realistic behaviours. The original MGN [11] used this algorithm in combination with their architecture to improve the quality of the predicted cloth, but this comes with a significant computational cost and was not used in subsequent works.

Articles focusing on virtual try-on (VTO) applications such as [14, 12] rely on more advanced pipelines. In their case, they use motion capture (MoCap) data (from the *CMU Graphics Lab Motion Capture Database*), which provides a skeletal representation of a human body. These motions are then converted to the *SMPL* (Skinned Multi-Person Linear) format to obtain a human mesh. Finally, they use ARCSim to generate synthetic cloth draped on the bodies.

## 2.2. Self-supervised and Unsupervised Learning

Different learning frameworks have been proposed to tackle the cloth simulation problem. The first one used in the original MGN paper [11] was a supervised learning approach that leveraged precomputed ground-truth cloth motion generated by the simulator. Their implementation uses a loss computed as the error between the predicted and ground-truth cloth meshes, where the predicted mesh is obtained by integrating the predicted vertex accelerations using Euler method. In practice this is a per-node  $L_2$  loss on decoder outputs (accelerations in our case) against simulator targets, sometimes with an additional  $L_2$  term on the sizing field when learned remeshing is enabled. Training is performed on a single prediction step but evaluated on long rollouts; some works also train on short auto-regressive rollout segments to reduce exposure bias and improve temporal stability.

More advanced methods, such as [7], use physics-based information, which corresponds to a semi-supervised learning setting, meaning that they add more physical constraints both to the loss functions and to the mesh features. For instance, they track the kinetic energy of each node and use it as an additional feature. They also incorporate this quantity into the loss to encourage energy conservation, which helps improve temporal stability. A typical formulation is:

$$L = L_{sup} + L_{energy} \quad \text{with} \quad L_{energy} = \sum_i (E_i - \bar{E})^2$$

and  $L_{sup}$  is the per-node  $L_2$  target on accelerations,  $E_i$  is the kinetic energy of node  $i$ , and  $\bar{E}$  is a target computed using the ground-truth position and velocity.

Finally, the last family of approaches relies on self-supervised learning. Instead of using ground-truth data in the loss, the model is trained using physics-inspired loss functions that enforce realistic cloth behaviour. HOOD [4] is a representative example: it learns dynamic garment motion purely from geometric constraints such as stretching, bending, and temporal consistency. Concretely, the loss is split as:

$$L = L_{inertia} + L_{static}$$

with:

$$L_{inertia} = \frac{1}{2\Delta t^2} (x - \hat{x})^\top M (x - \hat{x})$$

and  $L_{static} = L_{strain} + L_{bending} + L_{gravity} + L_{collision}$

using  $\hat{x} = x + v\Delta t$ , the explicit Euler prediction from the previous step.

The other terms are defined as followed using the Saint Venant-Kirchhoff model (definitions from SNUG [13]):

$$\begin{aligned} L_{\text{strain}} &= \sum_{f \in F} \mathcal{A}_f \left( \frac{\lambda}{2} \text{tr}(G_f)^2 + \mu \text{tr}(G_f^2) \right), \\ L_{\text{bending}} &= \sum_{e \in E} (\theta_e - \theta_e^0)^2, \\ L_{\text{gravity}} &= -Mg^\top \mathbf{x}, \\ L_{\text{collision}} &= k_{\text{col}} \max(\varepsilon - d(\mathbf{x}), 0)^3. \end{aligned}$$

Here,  $\lambda$  and  $\mu$  are the Lamé parameters of the material,  $G_f$  is the Green strain tensor associated with face  $f$ ,  $\theta_e$  and  $\theta_e^0$  denote the current and rest dihedral angles of edge  $e$ , and  $d(\mathbf{x})$  is the distance from point  $x$  to the collider. Similar ideas appear in several works [1, 5, 4], which incorporate collision penalties, regularisation terms, or local rigidity constraints to guide learning without explicit supervision. SENC [6] further augments the loss with explicit self-collision penalties derived from proximity-based world edges to improve cloth–cloth contact.

Training in these self-supervised settings is often auto-regressive (e.g. in HOOD [4] up to 5 rollout frames are used) to stabilise recurrent hidden states and better align the loss with rollout behaviour.

### 3. Challenges and Limitations

#### 3.1. Incomplete Collision Handling

Despite significant progress in learning based cloth simulation, robust collision handling and particularly self-collision, remains a challenge. Many neural simulator either ignore collisions entirely or rely on learning it from data, which provides no guarantees and often fails. For instance, HOOD [4] explicitly acknowledges that it does not model garment self-collisions, leading to visible artifacts especially when distant parts of the cloth interact. The authors note that in a fully self-supervised setting, interaction patterns between remote garment nodes are unknown, and naively introducing dense spatial connectivity, as done through proximity-based world edges in MGN [11], would be computationally prohibitive and still insufficient for resolving complex self-collisions. Similarly, skinning based approaches such as SNUG [13] do not address self-collisions, identifying this limitation as an important direction for future work.

Recent work such as SENC [6] directly targets self-collision by introducing a self-collision loss. While effective at reducing penetrations, this approach introduces substantial computational overhead due to remeshing and volume computation, and requires meshes that can be meaningfully closed. This is problematic for garments with complex topology. The authors further acknowledge that closure strategies can restrict valid garment motion and that more general penetration measures that do not rely on mesh closure and can operate on open or layered garments is necessary.

#### 3.2. Temporal Instability and Extrapolation to Unseen Dynamics

Another fundamental limitation of current neural cloth simulators is their limited temporal stability. Most approaches adopt an auto-regressive rollout formulation, where each predicted state is used as input for the next time step. In such settings, small prediction errors accumulate over time, leading to drift and instabilities. This effect is amplified by the fact that most training are done using a single step and evaluation uses long horizon rollout. HOOD [4] explicitly reports that its model may fail when

body motion velocities exceed those observed during training, suggesting that learned dynamics do not extrapolate reliably. On the same topic, MGN-RP [7] demonstrates improved generalisation across mesh shape and motion trajectories, but acknowledges that rollouts become unstable when motion speeds are higher than those seen during training, with the cloth lagging behind control handles and only recovering once the motion stops.

These observations indicate that generalisation across geometry does not imply robustness across dynamics. Even physics informed losses which uses kinetic energy or inertia are insufficient to fully stabilise long auto-regressive inference. Better results are obtained when rollout is introduced during the training, for instance MGN-RP [7] randomly performs rollout on the previous prediction instead of a new frame of data (increasing the probability as training goes on).

### 3.3. Scalability Limits and Supervision Requirements

While learned simulators often achieve real-time inference, their training pipelines remain difficult to scale due to heavy supervision and data generation requirements. Supervised and semi-supervised methods typically rely on large amounts of high-quality physics-based simulation data created using tools like ARCSim [10, 9], which is expensive to generate. For example, N-Cloth [5] states that generating synthetic training datasets can take several days of physics simulation and must be repeated for new cloth topologies. This fixed topology assumption limits scalability and practical deployment, particularly for applications involving diverse garments or dynamically changing meshes.

Self-supervised approaches such as HOOD [4] reduce reliance on ground truth simulation data, but they still require carefully designed physical loss functions. And they remain sensitive to coverage in the training distribution, especially for rare but critical events such as extreme wrinkles or collisions.

A promising direction is to use semi-supervised curricula, where supervised losses are gradually annealed in favour of physical constraints during training.

### 3.4. Trade-off Between Detail Preservation and Physical Guarantees

Finally, current methods face a persistent trade-off between preserving fine scale geometric detail and enforcing physically meaningful behaviour. GNNs are known to suffer from over-smoothing. This issue becomes more pronounced as the number of message-passing layers increases, which is often required for long-range interaction modelling. Multi scale and hierarchical GNNs such as BSMS-GNN [2] mitigate this issue by enabling long-range information propagation, but existing pooling strategies can introduce connectivity artifacts, incorrect edges or loss of local structure.

At the same time, while many approaches incorporate physics inspired losses to improve realism, they do not provide formal physical guarantees comparable to classical numerical solvers. N-Cloth [5] recognises that it cannot guarantee physical accuracy or collision free output. In the case of HOOD [4], they emphasise that despite using physically based optimisation, interactions not seen during training remain unresolved in self-supervised settings.

These architectural choices also have computational implications, as message passing over irregular graphs is not trivially parallelisable over multiple steps. Unlike classical physics simulators, where constraints can be processed in optimised batches on the GPU, GNN operations involve scattered memory accesses and irregular graph structures. This makes real-time performance difficult to achieve when the mesh resolution becomes too large. For this reason, existing models typically rely on moderately sized meshes, which restricts the visual fidelity and realism of the predicted deformations.



## Conclusion

In this report, we reviewed recent advances in physics-informed deep learning techniques for cloth simulation. Modern approaches increasingly rely on graph neural networks to model local physical interactions on mesh structures, and several extensions of the MGN architecture have demonstrated strong performance across a range of dynamic phenomena. At the same time, new learning paradigms such as self-supervised and physics-driven losses show promise in reducing reliance on explicit ground-truth supervision and improving the physical plausibility of predictions.

Despite these advances, several challenges remain. Current models still struggle with long-term temporal stability, and their ability to generalise to unseen velocities or motions. Training also depends heavily on the availability of large, high-quality synthetic datasets, which are costly to generate and may contain inaccuracies that propagate into the learned models. Finally, GNN-based simulators face computational constraints when scaling to high-resolution meshes, restricting their use in settings that require fine-grained detail or real-time performance.

Overall, physics-informed deep learning has shown significant potential for accelerating cloth simulation while maintaining realism, but further progress is needed to address these limitations. Improving generalisation capabilities, reducing dataset dependence, and enabling efficient computation on dense meshes remain key directions for future research.

## Acronyms

**FEM** finite element method. 1

**GNN** graph neural network. 1, 6, 7

**MGN** MeshGraphNet. 1, 2, 4–8

**VTO** virtual try-on. 4

## Glossary

**auto-regressive** Auto-regressive models are models that predict future values based on past values. 4–6

**mesh** In 3D computer graphics, a (polygon) mesh is a set of vertices, edges and faces that form the surface of an object. The faces consist of convex polygons, usually triangles for easy processing. 1, 2, 4, 6, 8

**motion capture (MoCap)** Motion Capture is the process of recording and analyzing human motion data. 4

**multi-layer perceptron (MLP)** A multi-layer perceptron is a neural network with multiple layers of interconnected neurons. 2

**rollout** In learning-based physical simulation, a rollout is the process of repeatedly applying a model over successive time steps to generate a full simulated trajectory. 1, 4–6, 8

**self-supervised** Self-supervised learning is a paradigm where supervision signals are derived automatically from the data itself, without requiring explicit ground-truth labels. 4, 6, 7

**skeletal representation** In computer animation, 3D models such as human body are represented as a mesh and a skeleton. The skeleton is a set of bones and joints which are connected together and influence the mesh position. 1

**skinning** Skinning is the process of putting a mesh on top of a skeleton. 1, 5

## A Results Replication

### A1. MeshGraphNets [11]

To evaluate the performance of MGN, we implemented a focused replication of the cloth-related experiments from the original paper [11]. Our replication targets only deformable cloth objects, as these are directly aligned with the goals of the report. Other types of simulations, such as fluid dynamics or rigid body dynamics, are beyond the scope of this report.

The model follows the architecture, including the encoder–processor–decoder structure. In contrast to the original implementation, we did not include the adaptive remeshing component. This decision was driven both by its complexity and by the fact that later works typically avoid remeshing while still achieving realistic cloth predictions.

In a first experiment, we did not use self-collision or collision with objects. With this restriction, we only needed mesh edges. Later on, we tried to include collisions, but the training process becomes much longer to achieve satisfactory results.

Training data was generated using a custom Python interface to ARCSim [10, 9], allowing us to create controlled cloth simulations with varying parameters and motion conditions. These datasets are smaller than those used in [11], but they capture sufficient dynamic variability to train and evaluate the model. Under these conditions, the replicated MGN implementation successfully learns to produce visually plausible cloth trajectories. However, as expected, the model exhibits drift during long rollout and struggles to reproduce fine-scale wrinkles. Despite these limitations, the replication provides a reliable baseline for further experimentation.

### A2. Experiments with Alternative Learning Methods

Beyond the supervised setup used in MGN, we explored several alternative learning strategies, including semi-supervised and self-supervised formulations. These approaches aim to reduce the reliance on high-quality ground-truth datasets and to embed additional physical meaning directly into the training objective.

Our semi-supervised experiments combine a standard supervised loss with physics-inspired regularization terms, such as the ones described in HOOD [4] and SNUG [13]. The first results of this experiment were not great as the gravity term always prevailed during training, making the cloth behave like a solid plank and stretching near the fixed points. To improve the results, we needed to add rollout during training. This was done by implementing the probabilistic approach of MGN-RP [7].

## References

- [1] Hugo Bertiche, Meysam Madadi, and Sergio Escalera. “Neural Cloth Simulation”. In: (2022). DOI: [10.1145/3550454.3555491](https://doi.org/10.1145/3550454.3555491). arXiv: [2212.11220](https://arxiv.org/abs/2212.11220).
- [2] Yadi Cao et al. *Efficient Learning of Mesh-Based Physical Simulation with BSMS-GNN*. 2022. arXiv: [2210.02573](https://arxiv.org/abs/2210.02573).
- [3] Ruochen Chen, Thuy Tran, and Shaifali Parashar. *FNOPT: Resolution-Agnostic, Self-Supervised Cloth Simulation using Meta-Optimization with Fourier Neural Operators*. 2025. arXiv: [2512.05762](https://arxiv.org/abs/2512.05762) [cs.CV]. URL: <https://arxiv.org/abs/2512.05762>.
- [4] Artur Grigorev et al. “HOOD: Hierarchical Graphs for Generalized Modelling of Clothing Dynamics”. In: (2022). arXiv: [2212.07242](https://arxiv.org/abs/2212.07242).
- [5] Yudi Li et al. “N-Cloth: Predicting 3D Cloth Deformation with Mesh-Based Networks”. In: *Computer Graphics Forum (Proceedings of Eurographics)* 41.2 (May 2022), pp. 547–558.
- [6] Zhouyingcheng Liao, Sinan Wang, and Taku Komura. *SENC: Handling Self-collision in Neural Cloth Simulation*. 2024. arXiv: [2407.12479](https://arxiv.org/abs/2407.12479).
- [7] Emmanuel Ian Libao et al. “MeshGraphNetRP: Improving Generalization of GNN-based Cloth Simulation”. In: *ACM SIGGRAPH Conference on Motion Interaction and Games*. MIG ’23. ACM, Nov. 2023, pp. 1–7. DOI: [10.1145/3623264.3624441](https://doi.org/10.1145/3623264.3624441). URL: <http://dx.doi.org/10.1145/3623264.3624441>.
- [8] Mohammad Amin Nabian et al. “X-MeshGraphNet: Scalable Multi-Scale Graph Neural Networks for Physics Simulation”. In: *ACM SIGGRAPH Conference on Motion Interaction and Games*. MIG ’23. ACM, Nov. 2024, pp. 1–7. DOI: [10.1145/3623264.3624441](https://doi.org/10.1145/3623264.3624441). URL: <http://dx.doi.org/10.1145/3623264.3624441>.
- [9] Rahul Narain, Tobias Pfaff, and James F. O’Brien. “Folding and Crumpling Adaptive Sheets”. In: *ACM Transactions on Graphics* 32.4 (July 2013). Proceedings of ACM SIGGRAPH 2013, Anaheim, 51:1–8. URL: <http://graphics.berkeley.edu/papers/Narain-FCA-2013-07/>.
- [10] Rahul Narain, Armin Samii, and James F. O’Brien. “Adaptive Anisotropic Remeshing for Cloth Simulation”. In: *ACM Transactions on Graphics* 31.6 (Nov. 2012). Proceedings of ACM SIGGRAPH Asia 2012, Singapore, 147:1–10. URL: <http://graphics.berkeley.edu/papers/Narain-AAR-2012-11/>.
- [11] Tobias Pfaff et al. “Learning Mesh-Based Simulation with Graph Networks”. In: (2021). arXiv: [2010.03409](https://arxiv.org/abs/2010.03409) [cs.LG]. URL: <https://arxiv.org/abs/2010.03409>.
- [12] Igor Santesteban, Miguel A. Otaduy, and Dan Casas. “Learning-Based Animation of Clothing for Virtual Try-On”. In: *Computer Graphics Forum (Proc. Eurographics)* (2019). ISSN: 1467-8659. DOI: [10.1111/cgf.13643](https://doi.org/10.1111/cgf.13643).
- [13] Igor Santesteban, Miguel A. Otaduy, and Dan Casas. *SNUG: Self-Supervised Neural Dynamic Garments*. 2022. arXiv: [2204.02219](https://arxiv.org/abs/2204.02219).
- [14] Igor Santesteban et al. “Self-Supervised Collision Handling via Generative 3D Garment Models for Virtual Try-On”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021).